MINISTRY OF EDUCATION, CULTURE AND RESEARCH OF THE REPUBLIC OF MOLDOVA

Technical University of Moldova

Faculty of Computers, Informatics and Microelectronics

Department of Software Engineering and Automation

REPORT

Laboratory Works No.5 and No.6

The File System with Your Own Hands

Inside the xv6 File System

Operating Systems

Student:
Tacu Igor
FAF-242

Checked by:
Reițman Patricia, Universitary Assistant

UTM Chișinău – 2026

Purpose of the Laboratory Works

The purpose of Laboratory Work No.5 is to investigate the Linux file-system interface through direct experiments with paths, file types, inodes, links, permissions, storage allocation, mounted devices, and the /proc pseudo-file system. Laboratory Work No.6 continues the same study inside xv6, relating user-visible operations to on-disk structures and kernel code, and implementing two small utilities: mystat and find.

Objectives

1. Distinguish regular files, directories, symbolic links, character devices, and block devices. 2. Explain inode identity, link counts, symbolic-link behavior, directory entries, and sparse-file allocation.
3. Interpret Linux mode bits, umask, directory permissions, set-user-ID, and the sticky bit. 4. Inspect disks, mounts, CPU, memory, process metadata, and file descriptors. 5. Examine xv6 stat, dinode, directory-entry, unlink, and block-reclamation logic. 6. Implement and validate xv6 user programs that display metadata and recursively search directories.

Experimental Environment

The Linux experiments shown here were run on Ubuntu as root. The file-type commands were executed in /root/os-lab5; later tests were repeated in /root/os-lab5-repeat. Because root bypasses ordinary permissions, the permission screenshots do not demonstrate access denial for an ordinary user. xv6-riscv was run in QEMU.

Report Structure

Part I presents the Linux experiments required by Laboratory Work No.5. Part II moves inside xv6 for Laboratory Work No.6. The appendices contain the complete source code of the two implemented utilities, and the report ends with a joint critical analysis and conclusion.

1. Laboratory Work No.5 — The File System with Your Own Hands

1.1 Paths and File Types

Linux exposes several object types through one pathname interface. The first character printed by ls -l identifies the type: d for directory, - for regular file, l for symbolic link, c for character device, and b for block device. The /bin pathname is a symbolic link to /usr/bin on this Ubuntu system; /dev/null is a character device; and /tmp is a directory with the sticky bit set.

```
root@ubuntu-4gb-hel1-1:/# mkdir -p ~/os-lab5
root@ubuntu-4gb-hel1-1:/# cd ~/os-lab5
root@ubuntu-4gb-hel1-1:~/os-lab5# whoami
root
root@ubuntu-4gb-hel1-1:~/os-lab5# pwd
/root/os-lab5
root@ubuntu-4gb-hel1-1:~/os-lab5# printf '\n=== L5-01 FILE TYPES ===\n'

=== L5-01 FILE TYPES ===
root@ubuntu-4gb-hel1-1:~/os-lab5# ls /
app  bin.usr-is-merged  cdrom  etc   lib    lib.usr-is-merged  media  opt      proc  run   sbin.usr-is-merged  srv  tmp  var
bin  boot               dev    home  lib64  lost+found         mnt    os-lab1  root  sbin  snap                sys  usr
root@ubuntu-4gb-hel1-1:~/os-lab5# ls -ld /etc /etc/passwd /bin /dev/null /tmp
lrwxrwxrwx   1 root root     7 Apr 22  2024 /bin -> usr/bin
crw-rw-rw-   1 root root  1, 3 Sep 28 13:52 /dev/null
drwxr-xr-x 119 root root 12288 Oct 24 06:49 /etc
-rw-r--r--   1 root root  1798 Oct 12 12:41 /etc/passwd
drwxrwxrwt  11 root root  4096 Oct 26 11:24 /tmp
root@ubuntu-4gb-hel1-1:~/os-lab5# file /usr/bin/ls /etc/passwd /bin
/usr/bin/ls: ELF 64-bit LSB pie executable, x86-64, version 1 (SYSV), dynamically linked, interpreter /lib64/ld-linux-x86-64.so.2, BuildID[sha1]=44900829b39271d878615772bb2a6ffa4
22f0891, for GNU/Linux 3.2.0, stripped
/etc/passwd: ASCII text
/bin:        symbolic link to usr/bin
root@ubuntu-4gb-hel1-1:~/os-lab5# readlink -f /bin
/usr/bin
root@ubuntu-4gb-hel1-1:~/os-lab5# printf '\nCharacter device example:\n'
find /dev -maxdepth 1 -type c -print | head -1

Character device example:
/dev/zfs
root@ubuntu-4gb-hel1-1:~/os-lab5# printf '\nBlock device example:\n'
find /dev -maxdepth 1 -type b -print | head -1

Block device example:
/dev/sr0
root@ubuntu-4gb-hel1-1:~/os-lab5#
```

Figure 1: Linux path types and examples of character and block devices

Observation

A character device transfers a stream of bytes, while a block device is addressed in blocks and normally backs a file system. Symbolic links are distinct objects whose contents are pathnames; therefore /bin and /usr/bin refer to the same directory after pathname resolution.

1.2 Directory Size and Disk Usage

The size shown for a directory entry is metadata storage, not the recursive size of its contents. The du command traverses the hierarchy and totals allocated blocks. In the new measurement, /var/log used about 1.3 GiB and /usr used about 3.5 GiB, making /usr about 2.2 GiB larger.

```
$ ls -ld /var/log /usr
$ du -sh /var/log /usr
```

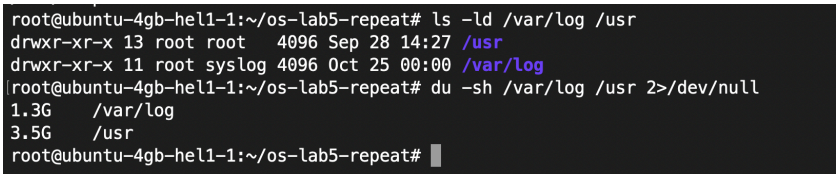


Figure 2: Directory metadata size compared with recursive disk usage

1.3 Inodes and Hard Links

An inode stores metadata and block addresses; a directory maps names to inode numbers. The names notes.txt and hard.txt shared inode 255785 after the hard link was created. Its link count rose from one to two. Removing notes.txt reduced the count, while hard.txt continued to refer to the same data.

```
$ echo hello > notes.txt
$ ls -li notes.txt
$ ln notes.txt hard.txt
$ ls -li notes.txt hard.txt
$ rm notes.txt
$ cat hard.txt
```

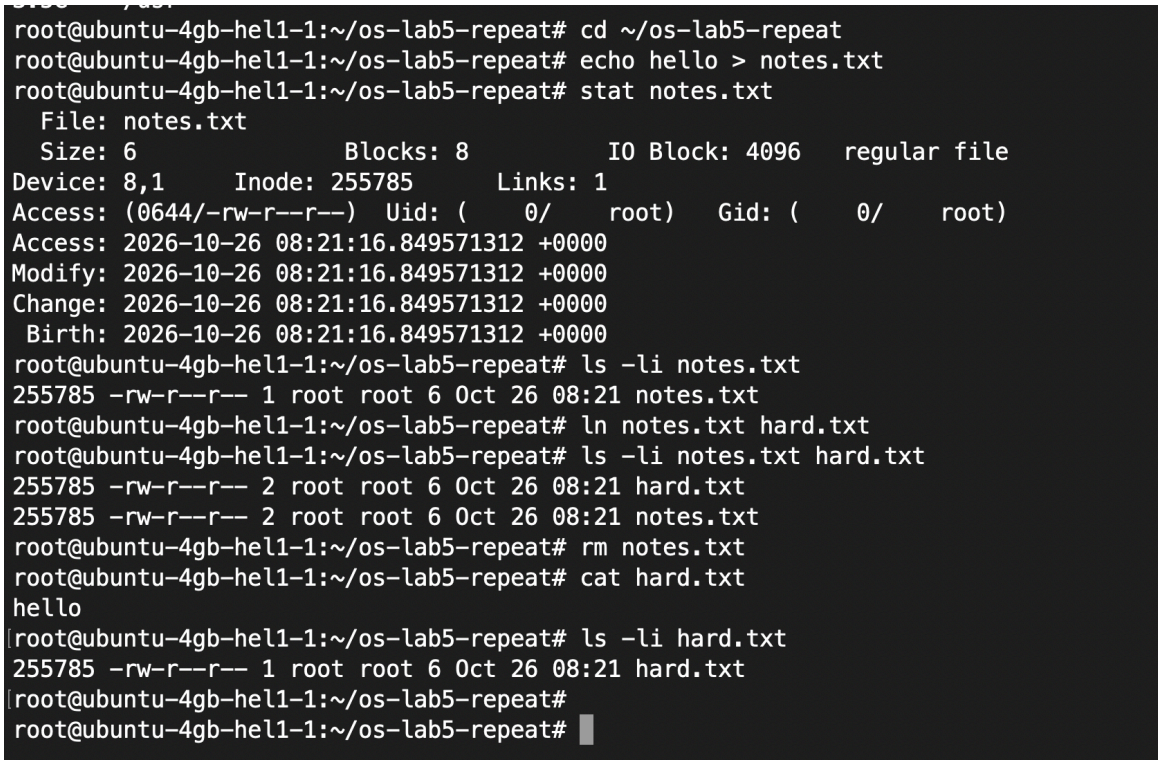


Figure 3: One inode referenced by two hard links

1.4 Symbolic Links and Directory Link Counts

The symbolic link soft.txt had its own inode and stored the eight-character target path hard.txt. Once hard.txt was removed, dereferencing soft.txt failed even though the symbolic-link object remained. A newly created directory had link count two for its parent entry and its own dot entry. Adding subdirectories x and y increased the parent directory's count from two to four because each child contains a .. entry referring to the parent.

```
root@ubuntu-4gb-hel1-1:~/os-lab5-repeat# ln -s hard.txt soft.txt
root@ubuntu-4gb-hel1-1:~/os-lab5-repeat# ls -li hard.txt soft.txt
255785 -rw-r--r-- 1 root root 6 Oct 26 08:21 hard.txt
256217 lrwxrwxrwx 1 root root 8 Oct 26 08:23 soft.txt -> hard.txt
root@ubuntu-4gb-hel1-1:~/os-lab5-repeat# cat soft.txt
hello
root@ubuntu-4gb-hel1-1:~/os-lab5-repeat# rm hard.txt
root@ubuntu-4gb-hel1-1:~/os-lab5-repeat# cat soft.txt
cat: soft.txt: No such file or directory
root@ubuntu-4gb-hel1-1:~/os-lab5-repeat# ls -l soft.txt
lrwxrwxrwx 1 root root 8 Oct 26 08:23 soft.txt -> hard.txt
root@ubuntu-4gb-hel1-1:~/os-lab5-repeat# mkdir d
root@ubuntu-4gb-hel1-1:~/os-lab5-repeat# ls -ld d
drwxr-xr-x 2 root root 4096 Oct 26 08:23 d
root@ubuntu-4gb-hel1-1:~/os-lab5-repeat# mkdir d/x d/y
root@ubuntu-4gb-hel1-1:~/os-lab5-repeat# ls -ld d
drwxr-xr-x 4 root root 4096 Oct 26 08:23 d
root@ubuntu-4gb-hel1-1:~/os-lab5-repeat#
```

Figure 4: Broken symbolic link and directory link-count growt

1.5 Logical Size versus Allocated Space

File length and allocated space are separate concepts. The two-byte file one.txt occupied one 4 KiB allocation unit. truncate created big.img with a logical length of 1 GiB but no data blocks, so du reported zero allocated space. This sparse representation is useful for disk images and large files containing extensive zero-filled regions.

```
$ echo x > one.txt
$ ls -l one.txt
$ du -h one.txt
$ truncate -s 1G big.img
$ ls -lh big.img
$ du -h big.img
```

```
root@ubuntu-4gb-hel1-1:~/os-lab5-repeat# echo x > one.txt
root@ubuntu-4gb-hel1-1:~/os-lab5-repeat# ls -l one.txt
-rw-r--r-- 1 root root 2 Oct 26 08:26 one.txt
root@ubuntu-4gb-hel1-1:~/os-lab5-repeat# du -h one.txt
4.0K    one.txt
root@ubuntu-4gb-hel1-1:~/os-lab5-repeat# truncate -s 1G big.img
root@ubuntu-4gb-hel1-1:~/os-lab5-repeat# ls -lh big.img
-rw-r--r-- 1 root root 1.0G Oct 26 08:26 big.img
root@ubuntu-4gb-hel1-1:~/os-lab5-repeat# du -h big.img
0       big.img
root@ubuntu-4gb-hel1-1:~/os-lab5-repeat#

root@ubuntu-4gb-hel1-1:~/os-lab5-repeat# stat -c 'logical size: %s bytes; allocated blocks: %b; block unit: %B bytes' big.img
logical size: 1073741824 bytes; allocated blocks: 0; block unit: 512 bytes
root@ubuntu-4gb-hel1-1:~/os-lab5-repeat#
```

Figure 5: Logical length and allocated space of a sparse file

1.6 File Permissions and umask

These checks were run as root. Mode 000 removed ordinary permission bits from s.txt, but root could still read it; this run therefore does not demonstrate denial for an unprivileged account. chmod u+r restored the owner read bit. With umask 0002, new files and directories received modes 664 and 775.

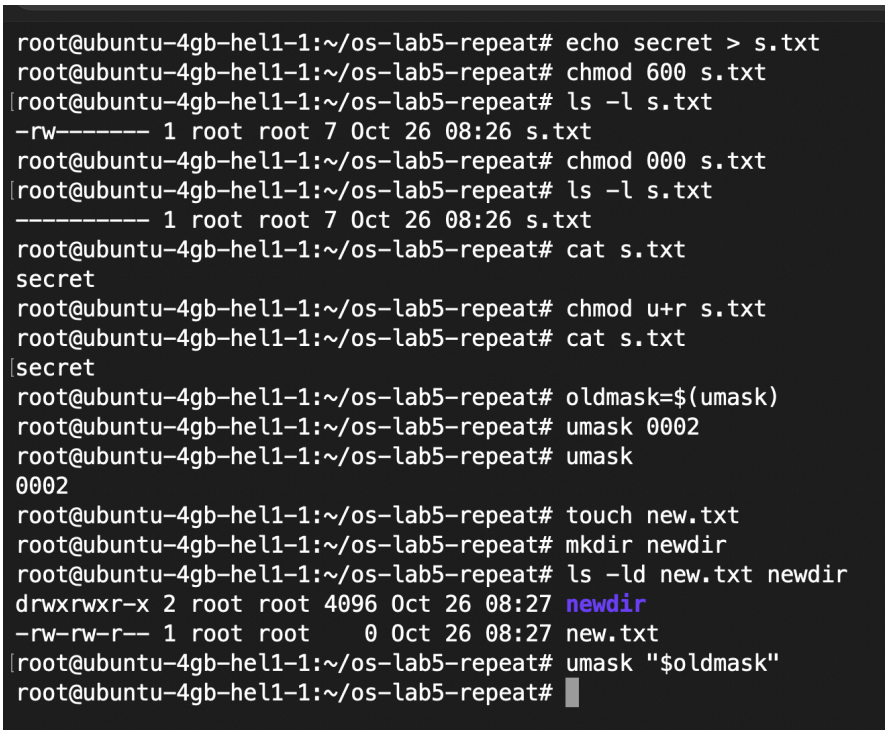


Figure 6: Permission changes as root and umask-derived modes

Recommended Modes

600 is appropriate for private keys and user-only secrets; 644 for ordinary documents readable by others; 700 for private directories or scripts; and 755 for programs and public directories that others must traverse. The least permissive mode that still supports the intended use should be selected.

1.7 Directory Permissions and Special Bits

Directory read permission permits listing names, while execute permission permits traversal. In this run, both ls box and cat box/f succeeded after restrictive mode changes because the commands ran as root. These results do not establish how an ordinary user would be affected.

The set-user-ID bit on /usr/bin/passwd permits operations with the file owner's effective identity. The sticky bit on /tmp limits deletion of other users' files. Root can read /etc/shadow; its contents are excluded from the figure to avoid exposing credential hashes.

```
root@ubuntu-4gb-hel1-1:~/os-lab5-repeat# mkdir box
root@ubuntu-4gb-hel1-1:~/os-lab5-repeat# echo hi > box/f
root@ubuntu-4gb-hel1-1:~/os-lab5-repeat# chmod u=r,go= box
root@ubuntu-4gb-hel1-1:~/os-lab5-repeat# ls -ld box
dr-------- 2 root root 4096 Oct 26 08:27 box
root@ubuntu-4gb-hel1-1:~/os-lab5-repeat# ls box
f
root@ubuntu-4gb-hel1-1:~/os-lab5-repeat# cat box/f
hi
root@ubuntu-4gb-hel1-1:~/os-lab5-repeat# chmod u=x,go= box
root@ubuntu-4gb-hel1-1:~/os-lab5-repeat# ls -ld box
d--x------ 2 root root 4096 Oct 26 08:27 box
root@ubuntu-4gb-hel1-1:~/os-lab5-repeat# ls box
f
root@ubuntu-4gb-hel1-1:~/os-lab5-repeat# cat box/f
hi
root@ubuntu-4gb-hel1-1:~/os-lab5-repeat# chmod 755 box
root@ubuntu-4gb-hel1-1:~/os-lab5-repeat# ls -l /usr/bin/passwd
-rwsr-xr-x 1 root root 64152 May 30  2024 /usr/bin/passwd
root@ubuntu-4gb-hel1-1:~/os-lab5-repeat# ls -ld /tmp
drwxrwxrwt 11 root root 4096 Oct 26 08:27 /tmp
```

Figure 7: Directory read/execute semantics, set-user-ID, and sticky bit

1.8 Disks and Mounts

The root filesystem was ext4 on /dev/sda1. lsblk showed 1.1 GiB available and 93% use; df -h / showed 38 GiB total, 35 GiB used, 1.1 GiB available, and 98% use. The tools account for reserved blocks differently. The FAT32 EFI partition on /dev/sda15 was mounted at /boot/efi.

```
root@ubuntu-4gb-hel1-1:~/os-lab5-repeat# lsblk -f
NAME    FSTYPE FSVER LABEL UUID                                 FSAVAIL FSUSE% MOUNTPOINTS
sda
├─sda1  ext4   1.0         64f6fb02-4fed-4836-a33b-86e8993afdfa    1.1G    93% /
├─sda14
└─sda15 vfat   FAT32       227C-A484                             251.9M     0% /boot/efi
sr0
root@ubuntu-4gb-hel1-1:~/os-lab5-repeat# df -h /
Filesystem      Size  Used Avail Use% Mounted on
/dev/sda1        38G   35G  1.1G  98% /
root@ubuntu-4gb-hel1-1:~/os-lab5-repeat# findmnt /
TARGET
  SOURCE    FSTYPE OPTIONS
/ /dev/sda1 ext4   rw,relatime
root@ubuntu-4gb-hel1-1:~/os-lab5-repeat# mount | grep ' on / '
/dev/sda1 on / type ext4 (rw,relatime)
root@ubuntu-4gb-hel1-1:~/os-lab5-repeat#
```

Figure 8: Block devices, root-file-system usage, and mount information

1.9 The /proc Pseudo-File System

The system exposed two processors and 3,911,572 kB of physical memory, with approximately 2.0 GiB available and no swap configured. /proc/self refers dynamically to the process reading it; consequently head observed its own status and its own file-descriptor table. Descriptors 0, 1, and 2 pointed to the terminal, while descriptor 3 referred to the directory being listed.

```
root@ubuntu-4gb-hel1-1:~/os-lab5-repeat# grep -c '^processor' /proc/cpuinfo
2
root@ubuntu-4gb-hel1-1:~/os-lab5-repeat# head -3 /proc/meminfo
MemTotal:        3911572 kB
MemFree:          729380 kB
MemAvailable:    2038588 kB
root@ubuntu-4gb-hel1-1:~/os-lab5-repeat# free -h
               total        used        free      shared  buff/cache   available
Mem:           3.7Gi       1.8Gi       723Mi       5.3Mi       1.5Gi       2.0Gi
Swap:             0B          0B          0B
root@ubuntu-4gb-hel1-1:~/os-lab5-repeat# head -5 /proc/self/status
Name:   head
Umask:  0022
State:  R (running)
Tgid:   2152338
Ngid:   0
root@ubuntu-4gb-hel1-1:~/os-lab5-repeat# ls -l /proc/self/fd
total 0
lrwx------ 1 root root 64 Oct 26 08:28 0 -> /dev/pts/0
lrwx------ 1 root root 64 Oct 26 08:28 1 -> /dev/pts/0
lrwx------ 1 root root 64 Oct 26 08:28 2 -> /dev/pts/0
lr-x------ 1 root root 64 Oct 26 08:28 3 -> /proc/2152390/fd
root@ubuntu-4gb-hel1-1:~/os-lab5-repeat#
```

Figure 9: CPU, memory, process status, and open file descriptors in /proc

Lab 6 Inode identity and recursive pathname search

Tacu Igor, FAF 242
Operating Systems, Technical University of Moldova
New xv6 reproduction on 9 November 2026

Scope and implementation

This experiment examines two different tasks: reading metadata for a supplied name and walking a directory tree to discover names. The complete implementations are included as `xv6/user/mystat.c` and `xv6/user/find.c`, together with their UPROGS entries and the full kernel tree needed to build them. The following results were collected from that tree in QEMU with one emulated hart, using a fresh generated filesystem image.

The tests use a disposable file named `specimen`; they do not remove the xv6 README. This preserves the teaching image while still testing what happens when a name is unlinked. The test tree has both files and directories whose basenames can match a search target, so successful recursion alone cannot hide a failure to report directories.

Reading metadata without confusing a name with an inode

`mystat` loops over its operands and calls `stat` for each one. It translates the three public xv6 types into file, directory, and device labels, and prints the inode number, link count, and byte size. If one lookup fails, it records failure and continues through later operands instead of aborting at the first bad name. The size uses the format for a 64-bit unsigned value rather than treating the field as a small integer.

The initial checks identified the root as directory inode 1 and console as a device. Writing `igor-file` plus its newline produced a ten-byte regular file at inode 33 in this image. Creating `alias` with `ln` made both names report the same inode and link count two. Removing only `specimen` left `alias` with count one and the same ten bytes, and `cat alias` still read the text. These observations separate three properties that are easy to conflate: the pathname used for lookup, the inode that holds identity and metadata, and the data still reachable through another name.

```
$ echo igor-file > specimen
$ mystat specimen
specimen: type=file inode=33 links=1 bytes=10
$ ln specimen alias
$ mystat specimen alias
specimen: type=file inode=33 links=2 bytes=10
alias: type=file inode=33 links=2 bytes=10
$ rm specimen
$ mystat alias
alias: type=file inode=33 links=1 bytes=10
$ cat alias
igor-file
$ mystat missing-path
mystat: cannot stat missing-path
```

Inode numbers depend on allocation order. The tested invariant is that both names identify one inode and that removing one name preserves the remaining link and contents.

A find implementation that also reports directories

The search utility takes a starting pathname and a basename. After stat it compares the current name before deciding whether to recurse, allowing directories themselves to match. It copies the

fixed-width `dirent.name` into a DIRSIZ-plus-one buffer and adds a terminator rather than assuming the kernel entry is already a C string.

The loop skips unused entries, dot, and dot-dot. Before constructing a child path, it checks the parent, separator, name, and terminator against its 512-byte buffer. Lookup and read failures produce diagnostics and a failure status. Every opened directory is closed after traversal. Ancestor descriptors remain open during recursion, so very deep trees are limited by xv6's descriptor limit.

Tests that distinguish traversal from correct matching

The tree contains `tree/target` and `tree/branch/target`, plus a directory `tree/branch/targetdir`. Searching for `target` printed the two files; searching for `targetdir` printed the directory; searching for `branch` printed the intermediate directory. A fourteen-character filename checked the DIRSIZ boundary, and a missing starting path exercised the diagnostic path. Invoking `find` without the two operands printed usage rather than interpreting uninitialized arguments.

```
$ mkdir tree
$ mkdir tree/branch
$ echo root-copy > tree/target
$ echo nested-copy > tree/branch/target
$ mkdir tree/branch/targetdir
$ find tree target
tree/branch/target
tree/target
$ find tree targetdir
tree/branch/targetdir
$ find tree branch
tree/branch
$ echo edge > tree/abcdefghijklmn
$ find tree abcdefghijklmn
tree/abcdefghijklmn
$ find missing-path target
find: cannot stat missing-path
$ find
usage: find start basename
$
```

These tests would expose a common bug in which basename comparison is performed only for regular files. Such a version can descend into `branch` and still fail to print `branch` or `targetdir`. The directory-target assertions in `scripts/run-xv6.py` verify that behavior explicitly. The raw console output is retained in `Lab5-6/evidence/xv6.txt`, so the displayed paths can be checked against the commands that constructed them.

Why unlink does not always free the blocks immediately

In `kernel/sysfile.c`, `sys_unlink` removes the name and updates link counts. In `kernel/fs.c`, `iput` waits for zero links and the final relevant in-memory reference before `itrunc` frees data blocks. An open reference can therefore preserve an unlinked inode.

The experiment proves survival through another hard link. The open-but-unlinked explanation comes from source inspection; it is a separate lifetime case.

Running `scripts/run-xv6.py` rebuilds the image and checks these distinctive link and search results. A fresh generated disk prevents leftover test names from affecting a rerun.